AutoGrader
A Rule-Driven Evaluator for C Programs

Assignment Report

Name: _____

Roll No.: _____

Course: _____

October 2026

Abstract

This report presents AutoGrader, an agent that grades student C programs without human intervention. Given a submission and an instructor-written rules file, it compiles and runs the program in a sandbox with fixed time and memory limits, evaluates nine metrics with independent analyzers (correctness, compiler warnings, structure, modularity, corner cases, time complexity, memory safety, readability and originality) and combines them into one weighted grade. For an incorrect program the feedback gives a concrete failing input with the expected output, the sanitizer diagnosis and an explanation. Readability is judged by an LLM on Groq, and a local Ollama `qwen3:4b` model proposes adversarial inputs, explains failures and writes reference solutions so that copying from an LLM can be detected. Originality is measured on an abstract structural representation of the code that ignores renaming, formatting and loop style. The system has a Django backend and a plain HTML/CSS frontend and runs on macOS and Linux. On ten sample submissions, each with a known flaw, the agent detected every flaw, and repeated grading on both platforms produced identical grades, metric scores and flags.

Contents

1 Introduction **2**
- 1.1 Objectives . . . 2
- 1.2 Summary of the solution . . . 2

2 System design **3**
- 2.1 Architecture . . . 3
- 2.2 Tools . . . 3
- 2.3 The rules file . . . 4

3 Methodology **4**
- 3.1 Safe execution . . . 4
- 3.2 The coordinator . . . 4
- 3.3 The analyzers . . . 5
- 3.4 Counter-test generation . . . 5
- 3.5 Complexity estimation . . . 6
- 3.6 Memory analysis and profiling . . . 6
- 3.7 Readability with an LLM . . . 6
- 3.8 Originality: abstract structural similarity . . . 6
- 3.9 Feedback generation . . . 7

4 Reproducibility **7**
- 4.1 Two noise sources found during testing . . . 8

5 Implementation **8**

6 Results **8**
- 6.1 The assignment's worked example . . . 8
- 6.2 All sample submissions . . . 9
- 6.3 Copy detection . . . 10
- 6.4 Reproducibility experiments . . . 10

7 Usefulness and limitations **11**
7.1 Usefulness 11
7.2 Limitations 11

8 Challenges and lessons learned **11**

9 Conclusion and future work **12**

A Running the agent **12**

B Project structure **13**

1 Introduction

When a class submits programs, someone has to run them, decide whether they are correct, how efficient they are and whether the code is written well. By hand this is slow, inconsistent between graders and does not scale. The task of this assignment is to build an *intelligent agent* that does the whole job on its own: it takes a C submission and the instructor's rules, runs the submission safely, checks it against every rule, and returns one overall grade with feedback that shows the student where the program falls short.

The central requirement is **reproducibility**: the same submission, judged under the same rules, must receive the same grade every time the agent runs. This is harder than it sounds, because two of the most useful signals are noisy: execution time varies with machine load, and LLMs do not always return the same answer. A large part of the design is about removing that noise without losing the signal.

1.1 Objectives

The agent must:

1. run the submission safely under fixed time and memory limits and survive programs that crash or never halt;
2. read the grading rules from a file outside its source code, so rules change without editing the agent;
3. evaluate each metric with its own analyzer and combine the results using the weights in the rules;
4. produce feedback that, for an incorrect submission, includes a failing input with the expected output, reports memory errors and leaks, and profiles the program;
5. flag suspected copying from peers, online resources or LLMs;
6. state the usefulness and limitations of the agent with concrete examples.

1.2 Summary of the solution

Objective	**How it is met**
1. Safe execution	Process-group sandbox with CPU-time, memory, output and core-dump limits; hangs, crashes and floods are caught (Section 3.1).
2. External rules	JSON rules files, validated on load and in the web editor (Section 2.3).
3. One analyzer per metric	Nine plug-in analyzers, dispatched by a coordinator and combined by weight (Sections 3.2, 3.3).
4. Feedback	Failing inputs from hidden tests and generated counter-tests, judged against a reference solution; ASan/UBSan; leak checks; CPU/memory profile; LLM tutor explanations (Section 3.9).
5. Copy flagging	Abstract structural fingerprints compared against peers and qwen3-generated solutions (Section 3.8).
6. Usefulness & limitations	Four of each with concrete cases, included in every report (Section 7).

Objective	How it is met
Reproducibility	Seeded generation, deterministic ordering, content-addressed LLM cache, CPU-time limits; verified experimentally on macOS and Linux (Section 4).

2 System design

2.1 Architecture

The system has two parts. The **engine** (`engine/`) is a framework-independent Python package containing the agent; it can be used from the command line. The **web application** (`grader/`, Django) stores submissions, queues them for a background worker and renders reports. Figure 1 shows the agent's pipeline.

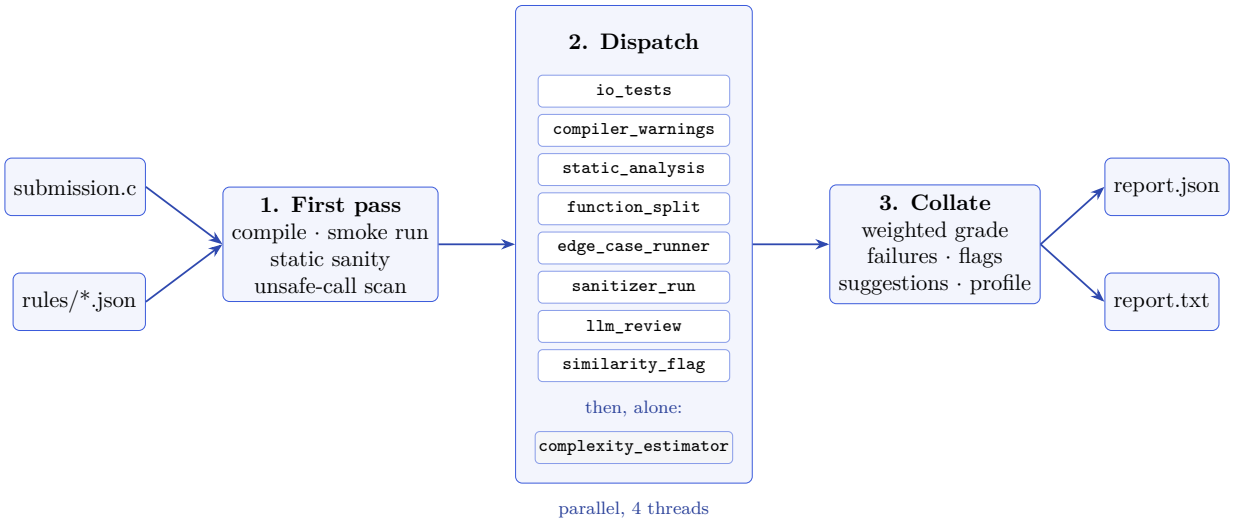


Figure 1: The agent's pipeline (`engine/agent.py`).

2.2 Tools

Tool	Used for
clang / gcc	Compilation, warnings, `-fsanitize=address,undefined`
AddressSanitizer, UBSan	Buffer overflows, use-after-free, undefined behaviour (e.g. signed overflow)
macOS `leaks`, LeakSanitizer, valgrind	Memory-leak detection with the leaking source line
lizard	Cyclomatic complexity and length per function
Groq (`openai/gpt-oss-120b`)	Readability judgement
Ollama (`qwen3:4b`, local)	Adversarial test inputs, tutor explanations, AI-written reference solutions
Django + SQLite	Web backend, submission history, background worker
HTML / CSS	Frontend (one hand-written stylesheet, light and dark)

2.3 The rules file

Rules are JSON files in `rules/`. Everything that defines a grading run is in the file: limits, compile flags, the input format, the reference solution, and the list of metrics, each with an analyzer name, a weight and parameters. The file `rules/add_two_integers.json` reproduces the assignment's example and adds the information the agent needs to generate its own tests:

```
"reference_solution": "problems/add_two_integers/reference.c",
"input_format": {"lines": [[{"name": "a", "type": "int"}, {"name": "b", "type": "int
    "}]]},
"metrics": [
  {"name": "correctness", "analyzer": "io_tests", "weight": 0.20, "params": {"
    testset_dir": "tests/add/"}},
  {"name": "logical", "analyzer": "edge_case_runner", "weight": 0.10,
   "params": {"cases": ["negatives", "zero", "int_overflow", "whitespace", "llm"]}},
  ...
]
```

The `input_format` describes stdin as lines of typed fields (`int`, `long`, `uint`, `float`, `word`) with optional ranges; a field can repeat as many times as another field's value, which describes arrays. The rules are validated on load: an unknown analyzer, a duplicate metric, a negative weight or malformed JSON is rejected with a clear message, also in the web editor. Weights that do not sum to 1 are normalised and the report notes it. A second rules file, `rules/sum_array.json`, defines a harder problem (sum of up to 10^6 integers, required $O(n)$), so that complexity, leak and copy detection have something real to detect.

3 Methodology

3.1 Safe execution

Every run of student code goes through `engine/sandbox.py`:

- **Time limit on CPU time.** The limit counts user + system CPU time, as on competitive-programming judges. `RLIMIT_CPU` kills spinning programs, and a wall-clock backstop of 3× the limit plus 1 s kills programs that hang without using CPU. The program runs in its own process group, which is killed as a whole.
- **Memory limit on resident memory.** The parent polls the program's memory every 2–10 ms (`libproc` on macOS, `/proc/<pid>/status` on Linux) and kills it above the cap; the peak from `wait4` is checked afterwards. On Linux, `RLIMIT_AS` at twice the limit is a safety net.
- **Output limit** (`RLIMIT_FSIZE`), no core dumps, a private working directory and a minimal environment.
- **Files instead of pipes** for stdin and stdout, so a program printing in an endless loop cannot deadlock the grader.
- **No dangerous calls.** Programs calling `system`, `fork`, `exec*`, `popen`, `socket`, `kill`, `unlink` and similar are never executed.

3.2 The coordinator

The coordinator (`engine/agent.py`) works in three phases.

First pass. A cheap sanity sweep: compile with `-O2 -Wall -Wextra -std=c11`, run the sample input under the limits, and check that `main` exists, input is read and output is written. If the smoke run times out, the report gets a `suspected_infinite_loop` flag; the execution-based analyzers (correctness, logic, complexity, memory safety) are skipped with score 0, the source-only analyzers still run, and a partial report is produced. A compile error or a dangerous call has the same effect.

Dispatch. Each metric is sent to its analyzer, in parallel on four threads. The complexity analyzer is timing-sensitive, so it runs alone afterwards. An exception in an analyzer becomes a score of 0 with the error recorded; it cannot break the report.

Collation. With normalised weights w_i and analyzer scores $s_i \in [0, 1]$:

$$\text{grade} = 100 \cdot \sum_i w_i \, s_i.$$

Results are combined in the order of the rules file, never in completion order. Failures from all analyzers are merged and de-duplicated, suggestions are ranked by the points their metric lost, and a profile is assembled.

3.3 The analyzers

Metric (analyzer)	Method	Score
correctness (`io_tests`)	Runs every hidden test, compares the output (whitespace-insensitive by default), records verdict, CPU time and peak memory per test.	passed / total
syntax (`compiler_warnings`)	Compiles with the rule's flags, counting warnings individually even under `-Werror`.	$1 - 0.1\times$warnings; 0 on errors
structure (`static_analysis`)	lizard cyclomatic complexity and function length against the limits; comments, header comment, single-block code, globals, `goto`, long lines, magic numbers.	$1 - \sum$ deductions
modularity (`function_split`)	Number of functions, helpers besides `main`, share of code in `main`, unused helpers.	1 if enough functions; 0.3 if only `main`
logical (`edge_case_runner`)	Generated corner cases and LLM counter-tests judged against the reference solution (Section 3.4).	passed / total
complexity (`complexity_estimator`)	Timing curve plus static loop analysis (Section 3.5).	1 if within bound, -0.5 per class above
memory_safety (`sanitizer_run`)	ASan + UBSan on every hidden test; leak check (Section 3.6).	$1 - 0.25\times$distinct issues
readability (`llm_review`)	Groq LLM with a fixed rubric (Section 3.7).	LLM score / 10
originality (`similarity_flag`)	Abstract structural similarity (Section 3.8).	1 below threshold; flagged above

3.4 Counter-test generation

The assignment asks the agent to generate counter test cases covering corner cases. The edge-case analyzer builds inputs from the rules' `input_format` in categories:

`zero`	all zero; zero mixed with other values; minimum size
`negatives`	-1, small and large negative values
`int_overflow`	[max], [min], [max, 1], [min, -1], [max, min] for full-range integers
`boundaries`	every field at its declared minimum and maximum
`whitespace`	the same data with tabs, extra spaces, blank lines, no trailing newline
`large`, `single`, `empty`	maximum size, $n = 1$, $n = 0$
`llm`	qwen3 reads the problem *and the student's code* and proposes inputs likely to break this program

The random generator for each category is seeded from a hash of the rules, so every run produces the same inputs. Every LLM-proposed input is validated against the input format before use; this was necessary because qwen3 often proposed, for example, a single number where two integers were required (2 of 5 proposals were rejected for `add.c`). Expected outputs come from the compiled reference solution, which acts as an oracle.

3.5 Complexity estimation

For each input size n in the rules, an input of exactly that size is generated, the program runs three times, and the *minimum* CPU time is kept, as the estimator least affected by noise. With the smallest size as a baseline, the slope of $\log(t_n - t_0)$ against $\log n$ is fitted, ignoring points below a 4 ms noise floor. Slope bands map to classes: below 0.35 is $O(1)$ or $O(\log n)$, below 1.35 is $O(n)$ or $O(n \log n)$, below 2.4 is $O(n^2)$, and above that $O(n^3)$. A static estimate (maximum loop nesting depth, recursion, use of `qsort`) breaks ties and is used when the timings are flat. A timeout at some size means the required bound is exceeded.

3.6 Memory analysis and profiling

The program is rebuilt with `-fsanitize=address,undefined -g` and run on every hidden test. Diagnostics are parsed into distinct *(kind, source line)* issues: heap, stack and global buffer overflows, use-after-free, double free and undefined behaviour. The source line is taken from the first stack frame inside the student's file. Leaks are detected by macOS `leaks` on macOS, and by LeakSanitizer (or valgrind, if installed) on Linux; all three report the leaking line. The profile lists the maximum CPU time, wall time and peak memory over the hidden tests, plus the scaling curve from the complexity analyzer.

3.7 Readability with an LLM

The code is sent to Groq with line numbers and a fixed 0–10 rubric covering names, comments, formatting, consistency and clarity; correctness is explicitly excluded. The request uses temperature 0, seed 42 and JSON output, and the answer provides the score, the reasons and line-specific suggestions.

The assignment's example uses `llama-3.3-70b-versatile`, but that model is no longer available on Groq for this account (HTTP 404, `model_not_found`). The available models were compared on `add.c`: `openai/gpt-oss-120b` gave 5/10 with accurate suggestions; `qwen/qwen3.8-27b` also gave 5/10 but claimed a missing space that was not missing. `openai/gpt-oss-120b` was chosen; it gives exactly the 5/10 that the assignment's example shows for this program.

3.8 Originality: abstract structural similarity

Comparing raw text is defeated by renaming variables or reformatting. AutoGrader compares programs on an abstract representation instead:

1. **Canonicalise:** `for (A; B; C) S` becomes `A; while (B) { S C; }`; braces are dropped, so optional braces do not matter; comments and whitespace are removed.

2. **Abstract:** identifiers become `V`; literals become `N`, `S` or `C`; integer types become `INT`; all loops become `LOOP`; `++`/`-` become `INC`; compound assignments become `OPEQ`. Library calls are kept, because they carry meaning.

3. **Compare** two views: token 5-grams (Jaccard similarity J) and the control-flow skeleton of loops, branches, returns and library calls (sequence alignment ratio S); similarity $= 0.75J + 0.25S$.

4. **Exclude base code:** 5-grams that also occur in the instructor's reference solution are removed first, so shared boilerplate is not evidence (as in MOSS).

5. **Corpus:** all peer submissions of the problem, plus three solutions written by qwen3 for the same problem statement, so copying from an LLM is detected the same way as copying from a peer.

6. **Size gate:** programs shorter than 50 abstract tokens are not flagged, because short programs such as a six-line $a + b$ are written identically by almost everyone.

A submission whose similarity reaches the threshold (0.85) is flagged as `suspected_copy` or `suspected_llm_copy` for human review, and its originality score falls linearly to 0 at similarity 1.

3.9 Feedback generation

Each failure is reported with the input, the expected output and the actual output, followed by three explanations: the **sanitizer** diagnosis from re-running that input under ASan/UBSan; a deterministic **WHY** (for example, detecting that the expected value lies outside the 32-bit range and the program's output differs from it by exactly 2^{32}, which means the value wrapped around); and an **AI tutor** note from qwen3 giving the root cause and the fix. Suggestions are ranked by how many points their metric lost. Each grading writes a JSON report for machines and a text report for the student.

Running qwen3 reliably. The installed `qwen3:4b` kept producing long reasoning text even with thinking disabled; one code-generation request took 70 s and returned no code. Requesting grammar-constrained JSON output (Ollama's JSON mode) makes it answer directly in about 4 s. Code is requested as an array of lines, because escaping a whole program into one JSON string triggered Ollama's repeat limit.

4 Reproducibility

The following mechanisms make the grade a function of the submission and the rules only:

- **Seeded inputs:** all generated inputs come from random generators seeded with a hash of the rules.
- **Deterministic order:** results are combined in rules order; all file and corpus listings are sorted.
- **LLM cache:** every LLM request is keyed by a hash of provider, model, messages, temperature and seed. The first answer is stored and replayed on every later run, and the cache is consulted before checking whether the service is reachable. Deterministic model failures are cached too, so a later successful retry cannot change a grade.

- **CPU-time limits** and minimum-of-N timing mapped onto discrete complexity classes; the web worker grades one submission at a time.
- **Audit trail:** each report stores hashes of the submission, the rules, and the rules + tests + reference solution, plus the corpus hash and tool versions.

4.1 Two noise sources found during testing

The LLM is not deterministic. Three fresh Groq requests with temperature 0 and seed 42 scored `sum_leak.c` 7, 8 and 7 out of 10. Without the cache, the overall grade would vary by 0.5 points between runs. The cache is therefore essential, not an optimisation.

Wall-clock time is not reliable. The first launch of a freshly compiled binary took 0.42 s of wall-clock time for 0.01 s of CPU time, because macOS scans new binaries. Under load, this once pushed the correct `add.c` past the 1 s limit, and it was flagged as an infinite loop and graded 40 instead of 75.72. Measuring the time limit in CPU time removed the problem.

5 Implementation

- **Web UI** (`grader/`): upload one or several `.c` files or paste code; a report page with the grade, a per-metric table with evidence, failure cards, suggestions, profile, the source code with flagged lines highlighted, and the usefulness and limitations section; submission history with statistics and a "clear all" button; a rules editor with validation.
- **Unattended use:** `python -m engine <files> -rules <rules>` or `manage.py grade` for batches, and an HTTP API (`POST /api/grade/`).
- **Cross-platform:** tested on macOS (Apple clang 21) and, in containers installed by the setup script, on Ubuntu 22.04 (Python 3.10, gcc 11), Debian 12 (Python 3.11, gcc 12), Ubuntu 24.04 (gcc 13) and Fedora 41 (gcc 14), plus x86_64 Ubuntu under emulation. Python 3.10 or newer is required. Running the code there exposed three problems, all fixed: memory peaks inherited from the grader process (solved with a small C launcher, `engine/launcher.c`); sanitizer reports without line numbers under gcc (solved with a shared parser); and cached LLM answers being ignored when Ollama was unreachable (solved by consulting the cache first). A setup script and Docker files handle installation. Testing on emulated x86_64 exposed one more problem: AddressSanitizer cannot run there, and the analyzer reported a false 10/10 for memory safety. The agent now self-tests the sanitizers first and, if they do not work, leaves the metric out of the grade with a flag instead of trusting silence.

6 Results

6.1 The assignment's worked example

Table 3 compares the agent's result for the assignment's `add.c` with the example in the assignment.

The failing case is reported as the assignment requires, with the expected output beside it:

Metric	Example	AutoGrader	Reason for any difference
correctness	18/20	16/20	8 of 10 tests pass in both; the example's 18/20 does not match its own "8 of 10", the agent scores passed/total
syntax	9/10	10/10	clang and gcc report no warning for this file; the example's `-Wunused-parameter` cannot occur, as `main` has no parameters
structure	6/10	7/10	no comments (−0.2) and a single block (−0.1)
modularity	3/10	3/10	only `main`
logical	8/10	7.2/10	13 of 18 corner cases; fails 4 of 5 overflow cases and 1 of 3 qwen3 cases
complexity	10/10	10/10	$O(1)$ confirmed
memory_safety	10/10	7.5/10	UBSan reports signed overflow at line 5 on hidden test 07: undefined behaviour is a real memory-safety finding
readability	5/10	5/10	single-letter names, no comments
originality	10/10	10/10	too short to judge; not flagged
grade	**89**	**75.72**	

Table 3: The assignment's example versus the agent's result for `add.c`.

```
Test 07  (correctness, WA)
   INPUT:     2147483647 1
   EXPECTED:  2147483648
   GOT:       -2147483648  [exit 0]
   SANITIZER: runtime error at line 5: signed integer overflow: 2147483647 + 1
              cannot be represented in type 'int'
   WHY:       The correct result does not fit in a 32-bit int, so the computation
              wrapped around (signed integer overflow is undefined behaviour in C).
              Use a wider type such as long long for the result.
   AI TUTOR:  The sum of 2147483647 and 1 exceeds the maximum value for a 32-bit
              signed integer, causing an overflow. Fix: Use a 64-bit integer type
              for the sum.
```

6.2 All sample submissions

Ten sample submissions were prepared, each with a known flaw (or none), to check that every metric detects what it should. Table 4 shows the results on macOS.

Table 4: Results on the sample submissions.

Submission	Planted flaw	Grade	Detected
`add.c`	assignment example: `int` sum	75.72	overflow on 2 hidden tests and 4 corner cases; UB at line 5; only `main`
`add_clean.c`	none	100.00	nothing to report
`add_infinite.c`	waits for a `.` that never arrives	40.00	`suspected_infinite_loop`; partial report
`add_syntax_error.c`	missing `;`	26.00	`compile_failed` with the line; source metrics still graded
`sum_good.c`	none (but copied by `sum_copy.c`)	89.50	`suspected_copy`, similarity 1.00
`sum_copy.c`	renamed, `for`→`while`, restyled copy	88.00	`suspected_copy`, similarity 1.00

Table 4: Results on the sample submissions.

Submission	Planted flaw	Grade	Detected
`sum_ai.c`	qwen3's solution, renamed and re-commented	85.50	`suspected_llm_copy`, similarity 1.00
`sum_quadratic.c`	correct but $O(n^2)$	79.50	slope 1.88 $\Rightarrow O(n^2)$; time limit at $n = 320\,000$
`sum_leak.c`	`int` accumulator; `malloc` never freed	77.55	5/8 tests; leak at line 12; UB at line 14
`sum_fixed_buffer.c`	`int buf[100]`, any n	82.64	6/8 tests; stack-buffer-overflow at line 6

Every planted flaw was detected by the metric meant to detect it, and the flawless `add_clean.c` scored 100. The complexity curve separates the algorithms clearly: `sum_good.c` took 3.4, 6.7 and 20.4 ms at $n = 20\,000$, 80 000 and 320 000 (slope 0.65, $O(n)$), while `sum_quadratic.c` went from 11.6 to 156 ms between the first two sizes (slope 1.88) and exceeded the limit at the third.

6.3 Copy detection

`sum_copy.c` renames every identifier of `sum_good.c`, replaces the `for` loop with a `while` loop, changes brace style and rewrites the comments; `sum_ai.c` does the same to qwen3's solution. Both reduce to exactly the same abstract stream as their source and are flagged at similarity 1.00. Before `for`/`while` canonicalisation and brace removal were added, the first pair scored only 0.57 and was missed. Independent solutions stayed well below the threshold: the highest similarity of `sum_quadratic.c` to any other program was 0.47.

An observation from the AI corpus: the three qwen3 solutions (seeds 1–3, temperature 0.7) differ as text but are structurally identical. A single generated solution would detect the same copies; more diverse references would need different prompts or models.

6.4 Reproducibility experiments

`scripts/check_reproducibility.py` grades every sample several times and compares the grade, every metric score and the set of flags.

Experiment	Platform	Result
9 samples × 3 runs, Groq enabled	macOS	identical
9 samples × 3 runs	Ubuntu 24.04 (gcc 13)	identical
both of the above at the same time (heavy load)	macOS + Linux	identical on both
`sum_ai.c` × 2 runs	macOS	identical (85.50)
web UI and HTTP API vs command line	macOS, Linux	identical (75.72 for `add.c`)
10 samples × 2 runs each on Ubuntu 22.04 and Debian 12	Python 3.10 / 3.11, gcc 11 / 12	identical; same grades as Ubuntu 24.04

Across platforms, 8 of 9 samples received exactly the same grade on Linux as on macOS. `sum_quadratic.c` scored 78.90 on Linux and 79.50 on macOS: its $O(n^2)$ algorithm is close to the

time limit on a 2-CPU virtual machine. This is expected, because time limits are a property of the machine; each machine is reproducible on its own.

7 Usefulness and limitations

7.1 Usefulness

1. **Large programming courses.** A 300-student lab is graded with one command. Every student is judged by the same rubric and receives the exact failing input (`2147483647 1`, expected `2147483648`) instead of a bare mark.

2. **Coding-interview screening.** For "sum an array in $O(n)$", a nested-loop solution is separated by the timing curve (`sum_quadratic.c` exceeds the limit at $n = 320\,000$) and an `int` accumulator by the overflow corner cases. Every rejection can be justified metric by metric.

3. **Plagiarism and LLM-copy triage.** Renamed and restyled copies of a peer (`sum_copy.c`) or of an LLM (`sum_ai.c`) are flagged at 1.00, so instructors review only the flagged submissions.

4. **Self-paced practice.** At any hour a student receives *memory leak at line 12*, a sanitizer trace and an explanation of the root cause, without staff time.

7.2 Limitations

Each limitation names a concrete case where the agent's verdict is wrong or unreliable.

1. **Tests cannot prove correctness.** A program that recognises the hidden inputs and prints their stored answers, or one that fails only for $n = 99\,991$, receives full correctness marks.

2. **Timing-based complexity is unreliable for small inputs.** With sizes $[1, 10, 100]$, an $O(n^2)$ program runs in under 1 ms at every size. The static fallback then counts `for (i = 0; i < 3; i++)` as an $O(n)$ loop and treats a recursive merge sort (two self-calls) as possibly exponential.

3. **Similarity cannot judge short programs and misses semantic rewrites.** A copied six-line $a + b$ receives full originality marks because of the size gate, and a copy rewritten from a loop into recursion is not flagged.

4. **Verdicts depend on the compiler and the machine.** `while (b != 0) { s += 1; }` never terminates, but clang `-O2` may delete this side-effect-free loop (C11 §6.8.5p6). We observed this: the program finished in 6 ms, and the agent reported a wrong answer instead of an infinite loop. Likewise, `sum_fixed_buffer.c` crashes for $n = 20\,000$ on Linux but not on macOS, because the default stack sizes differ.

A further caveat concerns readability: the cache makes re-grading identical, but clearing it or changing the model, as was forced when `llama-3.3-70b-versatile` became unavailable, can change the readability score for the same code.

8 Challenges and lessons learned

- **Platform differences are invisible until tested.** macOS ignores `RLIMIT_AS`; LeakSanitizer does not exist there; and on Linux the memory peak of the grader leaked into the

student program's statistics, making every test fail once the grader grew past 64 MB. Each was found only by running the code on the platform.

- **"Deterministic" LLM settings are not deterministic.** Temperature 0 with a fixed seed still varied between 7 and 8 out of 10. Reproducibility had to come from caching, not from model settings.
- **Small models need constraints.** qwen3 produced unusable output until JSON mode was forced, and its proposed inputs had to be validated against the input format.
- **Similarity needs domain knowledge.** Without canonicalising loops and braces a disguised copy was missed; without the size gate the assignment's own `add.c` was flagged as an LLM copy.

9 Conclusion and future work

AutoGrader meets all six objectives of the assignment. It runs untrusted C code safely, reads all grading decisions from an external rules file, evaluates nine metrics with independent analyzers, explains each failure with a concrete input, the expected output, a sanitizer diagnosis and a tutor note, flags copies of peers and of LLM output, and documents its own usefulness and limitations. Above all, it keeps the contract: repeated grading of every sample produced identical results on macOS and on Linux, including under heavy load.

Possible extensions are: property-based shrinking of failing inputs to the smallest counterexample; mutation testing of the hidden test set to measure how many bugs it can catch; container-level isolation (seccomp, no network) for high-stakes use; and calibrating time limits to the grading machine automatically by timing the reference solution.

A Running the agent

```
# setup (macOS)
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
echo 'GROQ_API_KEY=...' > .env && ollama pull qwen3:4b
.venv/bin/python manage.py migrate

# setup (Linux)
./scripts/setup_linux.sh        # or: docker compose up --build

# web UI
.venv/bin/python manage.py runserver 8000

# unattended grading
.venv/bin/python -m engine submissions/add_two_integers/add.c --rules rules/
    add_two_integers.json
.venv/bin/python manage.py grade submissions/sum_array/*.c --rules rules/sum_array.
    json --save

# HTTP API
curl -F file=@add.c -F rules=add_two_integers.json http://localhost:8000/api/grade/

# reproducibility check
.venv/bin/python scripts/check_reproducibility.py --runs 3
```

B Project structure

```
engine/          agent.py (coordinator), sandbox.py, launcher.c, context.py, inputs.py,
                 clexer.py, sanitizers.py, llm.py, explain.py, report.py, analyzers/
grader/          Django app: models, services (worker), views, templates, static CSS
rules/           add_two_integers.json, sum_array.json
problems/        statements and reference solutions
tests/           hidden tests (NN.in / NN.out)
submissions/     sample submissions; web uploads in submissions/<problem>/uploads/
corpus/          qwen3-generated reference solutions
reports/         generated JSON and text reports
scripts/         check_reproducibility.py, setup_linux.sh
docs/            this report and the detailed design document
```